

平衡树入门：FHQ Treap

一、Treap 是什么

二叉搜索树（BST）的规矩是「左子树都比我小，右子树都比我大」，中序遍历就是排好序的序列，一次查询的代价是树的高度。但按 1, 2, 3, 4, 5 顺序插入会得到一条链，高度 n ，退化成暴力。

Treap 的办法是给每个节点存两个值：

- *val*: 真正的数据，满足 **BST** 性质（左小右大）；
- *fix*: 一个随机优先级，满足大根堆性质（父亲的 *fix* 大于两个儿子）。

关键结论：*fix* 一定，树的形状就唯一确定。因为 *fix* 最大的必须当根，根定了以后 BST 性质把其余点分成左右两堆，各自再由堆里 *fix* 最大的当根，一路递推下去没得选。所以形状只和 *fix* 有关，与插入顺序无关，等价于「把这些数随机打乱后插进普通 BST」，期望高度 $O(\log n)$ 。出题人猜不到你的随机数，也就卡不掉。

FHQ Treap（非旋 Treap）是 Treap 的一种实现，好处是**不用写旋转**，只有 merge 和 split 两个操作，其余功能全靠这两个拼。

```
1 mt19937 gen(time(0));
2 struct Node{ int ls,rs,sz,fix,val; } t[N];
3 int cnt,rt;
4 int New(int v){ t[++cnt]={0,0,1,(int)gen(),v}; return cnt; }
5 void up(int p){ t[p].sz=t[t[p].ls].sz+t[t[p].rs].sz+1; }
```

sz 是子树里的点数，查第 k 小要用。0 号点当空节点，*sz* 恒为 0，省掉很多判空。

二、merge 与 split

merge（合并）

merge(x, y) 把两棵树接成一棵，前提是 x 里所有数都小于 y 里所有数。谁的 *fix* 大谁当根：若 $fix_x > fix_y$ 则 x 当根，它的左子树本来就最小不用动，递归把「 x 的右子树」和「整个 y 」合并后挂到右边；反之对称。

```
1 int merge(int x,int y){
2     if(!x||!y)return x+y;
3     if(t[x].fix>t[y].fix){ t[x].rs=merge(t[x].rs,y); up(x); return x; }
4     else { t[y].ls=merge(x,t[y].ls); up(y); return y; }
5 }
```

split（分裂）

split 是逆操作，把一棵树拆成两棵。按值拆： $\leq k$ 的进 x ， $> k$ 的进 y 。在根 p 处看：若 $val_p \leq k$ ，则 p 和它整个左子树都归 x ，令 $x = p$ 再去拆右子树；反之令 $y = p$ 去拆左子树。

```

1 void split(int p,int k,int &x,int &y){          // <=k 进 x, >k 进 y
2     if(!p){ x=y=0; return; }
3     if(t[p].val<=k) x=p, split(t[p].rs,k,t[p].rs,y), up(x);
4     else          y=p, split(t[p].ls,k,x,t[p].ls), up(y);
5 }

```

x, y 是引用：递归时直接把 $t[p].rs$ 传进去当 x ，子问题的结果会自动写回那个位置，不用手动接返回值。

按大小拆：最小的 k 个进 x 。把「比较 val 」换成「比较左子树点数」即可，往右递归时 k 要减掉左子树和自己：

```

1 void split(int p,int k,int &x,int &y){          // 前 k 个进 x
2     if(!p){ x=y=0; return; }
3     if(t[t[p].ls].sz+1<=k) x=p, split(t[p].rs,k-t[t[p].ls].sz-1,t[p].rs,y), up(x);
4     else          y=p, split(t[p].ls,k,x,t[p].ls), up(y);
5 }

```

两者都是每层往下走一步， $O(\log n)$ 。

三、拼出常见功能

套路固定：split 把要动的部分切出来，动完 merge 回去。

```

1 void insert(int v){                          // 插入
2     int a,b; split(rt,v,a,b);
3     rt=merge(merge(a,New(v)),b);
4 }
5 void erase(int v){                            // 删除一个 v
6     int a,b,c;
7     split(rt,v-1,a,b); split(b,v,b,c);      // a:<v  b:==v  c:>v
8     b=merge(t[b].ls,t[b].rs);                // 丢掉 b 的根
9     rt=merge(merge(a,b),c);
10 }
11 int kth(int p,int k){                        // 第 k 小, 不用 split
12     while(1){
13         int L=t[t[p].ls].sz;
14         if(k<=L) p=t[p].ls;
15         else if(k==L+1) return t[p].val;
16         else k-=L+1, p=t[p].rs;
17     }
18 }
19 int rnk(int v){                              // v 的排名
20     int a,b; split(rt,v-1,a,b);
21     int res=t[a].sz+1;
22     rt=merge(a,b);                            // 别忘了接回去
23     return res;
24 }

```

最重要的用法：维护序列

如果不存 val 、不管大小关系，只约定「中序遍历的顺序就是序列的顺序」，FHQ Treap 就成

了一个「能在任意位置切开、拼接的超级数组」。取区间 $[l, r]$ 只要切两刀：

```
1 int a,b,c;
2 split(rt,l-1,a,b);          // a=[1,l-1]
3 split(b,r-l+1,b,c);         // b=[l,r]   c=[r+1,n]
4 // b 这棵子树就是区间 [l,r]
5 rt=merge(merge(a,b),c);
```

再在节点上挂个「子树和」（在 `up` 里维护），切出来直接读就是区间和。这就是下面例题的全部基础：两次按大小 `split`，中间那棵树就是你要的区间。

四、例题：Z 博士的数列（series）

考察知识点：解线性递推、等比求和、逆元、平衡树维护区间赋值的块结构。

题意

数列满足 $25a_i + 20a_{i-1} = 12a_{i-2}$ 且所有 $a_i \geq 0$ ，给定 $a_0 = t$ 后唯一。长度 n 的缓冲区初始全 0，两种操作：1 l r t p 把 $a_p, a_{p+1}, \dots$ 依次写进位置 $l \sim r$ （覆盖）；2 l r 问 $l \sim r$ 的和，对 $10^9 + 9$ 取模。 $n \leq 10^9$ ， $m \leq 10^5$ ，强制在线（ l, r 要异或上次答案），内存 64 MB。

数列长什么样

猜 $a_i = x^i$ 代入，两边除以 x^{i-2} 得 $25x^2 + 20x - 12 = 0$ ，即 $(5x - 2)(5x + 6) = 0$ ，所以通解 $a_i = A\left(\frac{2}{5}\right)^i + B\left(-\frac{6}{5}\right)^i$ 。

用上 $a_i \geq 0$ ： $|\frac{6}{5}| > 1$ 会越来越大， $\left(\frac{2}{5}\right)^i$ 越来越小，所以只要 $B \neq 0$ ， i 大了以后第二项压倒第一项，而它正负交替，必有一半项为负，矛盾。故 $B = 0$ ，代入 $i = 0$ 得 $A = t$ ：

$$a_i = tq^i, \quad q = \frac{2}{5} = 2 \cdot 5^{-1}.$$

一次赋值写进去的是什么

位置 i 上放的是 $a_{p+i-l} = tq^{p+i-l}$ ，把与 i 无关的提出来，记 $K = tq^{p-l}$ ，则位置 i 的值就是 Kq^i 。这就是关键：一整段赋值区间只需记住一个数 K 。（ $p-l$ 可能为负，快速幂的指数要按费马小定理对 $\text{mod} - 1$ 取模。）这一段的和是等比求和

$$\sum_{i=l}^r Kq^i = K \cdot \frac{q^{r+1} - q^l}{q - 1},$$

除法即乘 $(q - 1)^{-1}$ ，只算一次存起来。

整个缓冲区长什么样

初始是「一整块 $K = 0$ 」，每次赋值就是把一段铲平换成新的一块。所以任何时刻缓冲区都是若干块的拼接，每块共用一个 K 。一次赋值最多把两端的块各切一半、中间的块全删、再加一块，块数每次最多加 2，总共 $O(m)$ 块——这就是 n 有 10^9 也能做的原因。

只维护块用 `map` 就够（珂朵莉树），但本题不行：赋值删块可以摊还，而询问不删块，一次询问可能横跨几万块又全留着， $O(m^2)$ 超时。所以要在树上维护子树和，切出区间直接读——用第三节最后那一招，节点存块：

字段	含义
K	本块系数，块内位置 i 的值是 Kq^i
val	本块长度
$val1$	本块内部的和
sz, sm	整棵子树的总长度、总和

和普通序列版的唯一区别：每个点算 val 个位置而不是 1 个，所以 `pushup` 里加 val ：

```

1 void pushup(int p){
2     t[p].sz=t[t[p].l].sz+t[t[p].r].sz+t[p].val;
3     t[p].sm=(t[t[p].l].sm+t[t[p].r].sm+t[p].val1)%mod;
4 }
5 int newnode(ll K,int l,int r){           // 建块时顺手算好等比和
6     int p=++cnt;
7     t[p].K=K; t[p].l=t[p].r=0;
8     t[p].sz=t[p].val=r-l+1;
9     t[p].val1=t[p].sm=(fpow(q,r+1)-fpow(q,l)+mod)*invq%mod*K%mod;
10    t[p].fix=gen();
11    return p;
12 }

```

`split` 要多带一个参数（本题唯一的难点）

麻烦在于切点可能落在某块中间，这时要把这块切成两块；而建块需要知道两半在全局的下标（ $val1$ 的公式里要用 l, r ），节点里却只存了长度。办法是递归时把「当前子树管的这段从哪开始」当参数 L 传下去。

```

1 // 把 rt 的前 sz 个位置切给 x，其余给 y；L=这棵子树在全局的左端点
2 void split(int rt,int sz,int &x,int &y,int L){
3     if(!rt){ x=y=0; return; }
4     if(t[rt].l.sz>=sz){           // 切点在左子树，起点不变
5         y=rt; split(t[y].l,sz,x,t[y].l,L); pushup(y);
6     }
7     else if(t[rt].l.sz+t[rt].val>sz){ // 切点落在 rt 自己这块中间
8         sz-=t[rt].l.sz;
9         L+=t[rt].l.sz;           // 此刻本块是 [L, L+val-1]
10        x=newnode(t[rt].K,L,L+sz-1); // 左半块
11        y=newnode(t[rt].K,L+sz,L+t[rt].val-1); // 右半块
12        t[x].l=t[rt].l; t[y].r=t[rt].r; // 原左右子树各自跟过去
13        pushup(x); pushup(y);
14    }
15    else{                           // 切点在右子树，起点要跳过左子树和自己
16        sz-=t[rt].l.sz+t[rt].val;
17        L+=t[rt].l.sz+t[rt].val;
18        x=rt; split(t[x].r,sz,t[x].r,y,L); pushup(x);
19    }
20 }

```

中间那段的下标最容易错，推一遍：设左子树长 lsz ，本块占全局 $[L+lsz, L+lsz+val-1]$ ，代

码已做 $L+=lsz$, 故此刻本块是 $[L, L + val - 1]$, 切前 sz 个给 x :

$$x = [L, L + sz - 1], \quad y = [L + sz, L + val - 1].$$

主过程

```
1 q=2*fpow(5)%mod; invq=fpow(q-1);
2 cin>>n>>m;
3 rt=cnt=1; t[1].sz=t[1].val=n; t[1].sm=0; t[1].fix=gen(); // 初始一块, K=0
4 for(int i=1;i<=m;i++){
5     int type; ll l,r,K,p; cin>>type>>l>>r;
6     l^=lastans, r^=lastans; // 强制在线, 先解密
7     int rt1,rt2,rt3;
8     split(rt,l-1,rt1,rt2,1); // rt1=[1,l-1]
9     split(rt2,r-1+1,rt2,rt3,1); // rt2=[l,r] rt3=[r+1,n]
10    if(type==1){
11        cin>>K>>p;
12        K=K*fpow(q,p-1)%mod; // t,p 归成一个 K
13        rt=merge(merge(rt1,newnode(K,l,r)),rt3); // 整段换成一块
14    }else{
15        lastans=t[rt2].sm%mod; // 直接读子树和
16        cout<<lastans<<"\n";
17        rt=merge(merge(rt1,rt2),rt3); // 原封不动接回去
18    }
19 }
```

第二刀的 L 要传 1, 因为 $rt2$ 管的是从位置 l 开始的那段; 第一刀传 1。

复杂度与空间

每次操作两次 split 加常数 merge, $O(m \log m)$ 。空间: 一次操作最多新建 5 个点 (两次 split 各切开一块, 赋值再建一块), $m = 10^5$ 即 5×10^5 个点, 每点 5 个 int 加 3 个 long long, 对齐后约 24 MB, 能过。卡得更紧就做节点回收: split 切开后原来那个点没人用了, 把编号丢进队列, newnode 优先从队列里取。

容易写错的地方

1. 右半块的右端点是 $L + val - 1$ 不是 $L + val$ 。多算一个位置, 块长和 sz 就不一致, 等比和也多一项。它不一定让样例挂: 必须恰好切在某块中间才触发, 小数据常常整块整块地切, 一到大数据就全错。
2. 数组开小。每次最多 5 个点, 要开到 5×10^5 以上。开 4×10^5 会在后半段越界, 症状是答案突然变 0 或 RE, 容易误判成算法错。
3. 忘了异或 lastans (初值 0, 第一次相当于没加密)。
4. 快速幂的负指数: q^{p-l} 中 $p-l$ 可能为负, 要写 $(y\%(\text{mod}-1)+(\text{mod}-1))\%(\text{mod}-1)$ 。
5. 取模减法补 +mod; 模数是 $10^9 + 9$ 不是 $10^9 + 7$ 。
6. 询问完别忘 merge 回去, 虽然没改数据, 但树已经被切成三段了。

7. 切块时给两个新点重随 fix ，理论上可能破坏堆序。这不影响**正确性**（split/merge 按 sz 走，不依赖堆序），只轻微影响平衡，随机之下无需处理。

小结

思路链条是：解递推 $\rightarrow$ 每段值形如 $Kq^i \rightarrow$ 区间赋值就是「换一块」 $\rightarrow$ 块数 $O(m) \rightarrow$ 询问要 $O(\log)$ 故维护子树和 $\rightarrow$ 平衡树维护块。平衡树在这里就是那个「能任意位置切开、还顺手告诉你区间和的超级数组」，掌握「两次 split 切出区间、动完 merge 回去」，剩下都是这题自己的数学。